# МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**

**НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**

**Факультет информационных технологий**

**Кафедра параллельных вычислений**

**ОТЧЕТ**

**О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ № 1**

«Два вектора»

студента 2 курса, 24204 группы

**Герасимова Алексея Александровича**

Направление 09.03.01 – «Информатика и вычислительная техника»

Преподаватель:

к. т. н.

А. Ю. Власенко

Новосибирск 2026

# СОДЕРЖАНИЕ

СОДЕРЖАНИЕ....2
ЦЕЛЬ....3
ЗАДАНИЕ....4
ОПИСАНИЕ РАБОТЫ....5
MPI....5
Последовательная программа....5
Параллельные программы....6
С использованием коммуникации типа точка-точка....6
С использованием коллективной коммуникации....8
Компиляция и запуск....10
Анализ результатов....10
Ускорение....11
Эффективность....12
ЗАКЛЮЧЕНИЕ....13
Приложение 1. Код последовательной программы....14
Приложение 2. Код программы с коммуникацией типа точка-точка....16
Приложение 3. Код программы с коллективной коммуникацией....20
Приложение 4. Скрипт для тестирования....24

# ЦЕЛЬ

Освоение принципов параллельного программирования с использованием MPI, реализация последовательного и параллельных алгоритмов вычисления суммы произведений элементов двух векторов, исследование времени выполнения, ускорения и эффективности параллельных программ при различном числе процессов.

# ЗАДАНИЕ

I. Написать 3 программы, каждая из которых рассчитывает число s по двум данным векторам a и b равной длины N в соответствии со следующим двойным циклом:

```
for (i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                s += a[i] * b[j];
```

a) последовательная программа

b) параллельная, использующая коммуникации типа точка-точка (MPI_Send, MPI_Recv)

c) параллельная, использующая коллективные коммуникации (MPI_Scatter, MPI_Reduce, MPI_Bcast)

II. Замерить время работы последовательной программы и параллельных на 2, 4, 8, 16, 24 процессах. Рекомендуется провести несколько замеров для каждого варианта запуска и выбрать минимальное время.

III. Построить графики времени, ускорения и эффективности.

# ОПИСАНИЕ РАБОТЫ

## MPI

MPI – Message Passing Interface (интерфейс передачи сообщений) – интерфейс для передачи информации между процессами, выполняющими одну задачу.

MPI – это стандарт – то есть документ, описывающий интерфейс и результат работы функций MPI. Реализаций стандарта может существовать сколько угодно. Примеры реализаций:

- MPICH – одна из самых первых свободная реализация MPI, работает на UNIX-системах и Windows NT
- Open MPI – ещё одна свободная реализация. Поддерживает множество ОС
- Intel MPI – коммерческая реализация для Windows и Linux. В лабораторной работе используется эта реализация.

Любую MPI программу нужно начинать с функции MPI_Init и заканчивать вызовом MPI_Finalize.

Каждый процесс может узнать свои rank и size – ранг процесса и количество процессов всего соответственно – с помощью MPI_Comm_rank(MPI_Comm cimm, int *rank) и MPI_Comm_size(MPI_Comm comm, int *size), где rank и size – выходные параметры. Во многих MPI функциях используются выходные параметры.

## Последовательная программа

Код последовательной программы на языке C находится в файле oneproc.c. (название обосновано тем, что фактически программа выполняется в единственном экземпляре — процессе). Полный листинг – в приложении 1.

Два вектора заполняются случайными числами следующим образом (листинг 1).

Листинг 1. Создание вектора

```
int* create_vector(int length) {
    int* vector = (int*) malloc(sizeof(int) * length);
    if (!vector) return NULL;
    for (int i = 0; i < length; i++) { vector[i] = rand() %
10; }
    return vector;
}
```

Несмотря на то, что программа последовательная, в неё используется MPI для функции MPI_Wtime (листинг 2), которая возвращает время, прошедшее от произвольного события в прошлом до текущего момента. С помощью неё измерена работа каждой из программ.

Листинг 2. Вычисление суммы и измерение времени

```
    const int length = atoi(argv[1]);

    int *x = create_vector(length);
    int *y = create_vector(length);

    double timer = MPI_Wtime();

    long long result = calculate(x, y, length);

    timer = MPI_Wtime() - timer;
    printf("%lld %.2lf\n", result, timer);
```

Фукция calculate(int*, int*, int) содержит в себе код из задания.

## Параллельные программы

### С использованием коммуникации типа точка-точка

Для передачи сообщений с использованием коммуникаций типа точка-точка в MPI есть функции MPI_Send и MPI_Recv. 1-ю функцию вызывает процесс-отправитель, 2-ю – получатель. Для того, чтобы получить именно то, что нужно, в MPI_Recv можно указать ранг отправителя и метку (для MPI_Send ранг

получателя и метку). Метка должна быть одной и той же при приеме и передаче сообщения.

Полный код программы с коммуникацией типа точка-точка можно найти в приложении 2.

Алгоритм следующий:

1. Процесс ранга 0 создаёт векторы
2. Отправляет весь 2-й вектор остальным процессам
3. Отправляет части 1-го вектора остальным процессам
4. Каждый процесс считает свою часть суммы
5. Все процессы отправляют процессу нулевого ранга свои результаты.
6. Процесс ранга 0 выводит складывает полученные суммы и выводит результат в стандартный поток вывода.

Листинг 3. Код, исполняемый процессом ранга 0.

```
    // init vectors
    int* x = create_vector(length);
    int* y = create_vector(length);

    // --------------- send vectors ---------------------//
    for (int i = 1; i < size; i++) {
        MPI_Send(y, length, MPI_INT, i, TAG_FULL_SECOND,
MPI_COMM_WORLD);
        int length_send = part_length(length, i, size);
        MPI_Send(x + i*x_part_length, length_send, MPI_INT,
i, TAG_PART_OF_FIRST, MPI_COMM_WORLD);
    }
    // -------------- calculate the part ----------------//
    long long sum = calculate(x, y, x_part_length, length);

    // ---------------- merge (recieve) results
-------------------//
    for (int i = 1; i < size; i++) {
        long long recieve;
        MPI_Recv(&recieve, 1, MPI_LONG_LONG, i, TAG_RESULT,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        sum += recieve;
    }
```

Листинг 4. Код, исполняемый процессами ранга не 0.

```
    // --------------- recieve vectors -------------------//
        // ---------------- 2nd vector -------------------
        int* y = (int*) malloc(sizeof(int) * length);
        MPI_Recv(y, length, MPI_INT, 0, TAG_FULL_SECOND,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        // ---------------- 1st vector -----------------
        int* x = (int*) malloc(sizeof(int) * x_part_length);
        MPI_Recv(x, x_part_length, MPI_INT, 0,
TAG_PART_OF_FIRST, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        // ----------------- calculate
-------------------- //
        long long sum = calculate(x, y, x_part_length,
length);
        // --------------- merge to rank0
----------------- //
        MPI_Send(&sum, 1, MPI_LONG_LONG, 0, TAG_RESULT,
MPI_COMM_WORLD);
        free(x);
```

**С использованием коллективной коммуникации**

Полный код программы можно найти в приложении 3.

Коллективные операции: MPI_Scatter, MPI_Bcast, MPI_Reduce

Особенность коллективной коммуникации заключается в том, что все процессы должны вызывать одну и ту же функцию. Только в этом случае осуществляется передача сообщений (в коммуникации типа точка-точка один процесс вызывал MPI_Send, другие – MPI_Recv).

MPI_Scatter разделяет данные с одного процесса на все. Например, если есть массив из 10 элементов, то его можно разослать его на 5 процессов (включая отправителя) – каждый будет иметь по 2 элемента. Проблема заключается в том, что размер буфера для каждого процесса должен быть одинаковый. Поэтому нужно будет отдельно посчитать остаток вектора, который не был отправлен во все процессы.

Чтобы работать с векторами, размер которых не кратен числу процессов, можно использовать MPI_Scatterv (и другие -v функции). В них также в качестве параметра передаётся смещения данных от каждого процесса и количества,

которое нужно передавать. Например, если нужно передать массив из 10 элементов 3-м процессам, то массив смещений будет [0, 3, 3], а массив количеств – [3, 3, 4]. 3-й процесс получит 4 элемента.

Для отправки 1-го вектора массивы заданы следующим образом (листинг 5): данные распределяются равномерно между всеми процессами, но процесс ранга 0 получит сверх этого остаток. Смещение вычисляется как смещение для предыдущего (по рангу) процесса + число отправленных элементов этого же процесса.

Листинг 5. Массивы смещений и количеств

```
int* displs = (int*) malloc(sizeof(int) * size);
// displs (англ. displaces) - смещения
int* sendcounts = (int*) malloc(sizeof(int) * size);

displs[0] = 0;
sendcounts[0] = length / size + length % size;
for (int i = 1; i < size; i++) {
    displs[i] = displs[i - 1] + sendcounts[i - 1];
    sendcounts[i] = length / size;
}
```

Алгоритм программы следующий:

1. MPI_Bcast отправляет 2-й вектор (global_y) всем остальным процессам.
2. MPI_Scatterv рассылает 1-й (global_x → local_x) вектор с учётом заданных ранее sendcounts и displs.
3. Происходит вычисление локальной суммы в каждом процессе
4. MPI_Reduce соединяет local_sum с global_sum в процессе с рангом 0. Как именно local_sum будт взаимодействовать с global_sum определяется бинарной функцией (4 параметр). Здесь используется MPI_SUM.
5. Результат выводится на экран процессом ранга 0.

Листинг 6. Использование коллективных операций

```
 int* local_x = (int*) malloc(sizeof(int) *
sendcounts[rank]);
    MPI_Bcast(global_y, length, MPI_INT, 0, MPI_COMM_WORLD);

    MPI_Scatterv(global_x, sendcounts, displs, MPI_INT,
        local_x, sendcounts[rank], MPI_INT, 0,
MPI_COMM_WORLD);

    long long local_sum = calculate(local_x, global_y,
sendcounts[rank], length);
    long long global_sum = 0;
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_LONG_LONG,
MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        timer = MPI_Wtime() - timer;
        printf("%lld %.2lf\n", global_sum, timer);
    }
```

## Компиляция и запуск

Для компиляции был написан скрипт test_all.sh (листинг 7), в котором для каждого .c файла запускается mpiicx – программа, которая правильно компонует исходный код с реализацией mpi. Для компиляции кода на языке C используется компилятор icx от Intel.

Листинг 7. Скрипт для сборки всех версий программ

```
#!/bin/bash
for file in *.c; do
    mpiicx "$file" -o "${file%.c}.out"
done
```

Для проведения измерений также был написан скрипт. Его можно найти в приложении 4. В качестве параметров он принимает длину вектора и число тестов. Результаты сохраняются в файлы вида log_X.txt, где X – номер теста.

## Анализ результатов

Было проведено 3 измерения для N=400000. В каждом из измерений выбиралось минимальное время. Получены следующие результаты.

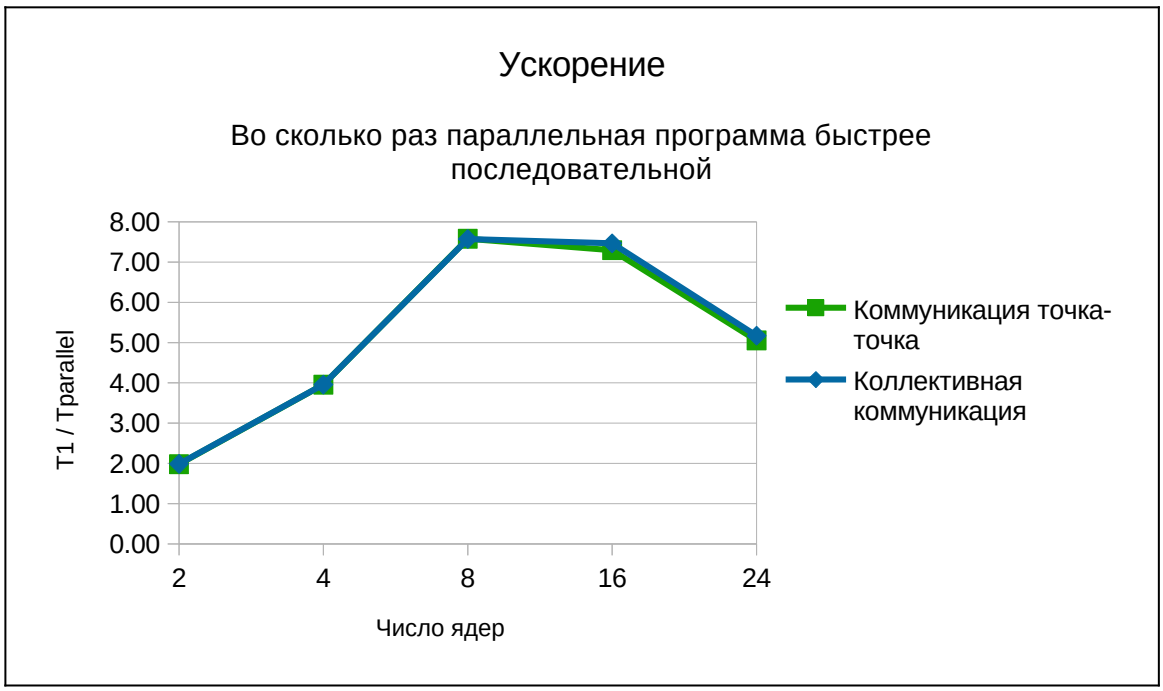
Таблица 1: Время работы всех программ

| Количество ядер | Время работы программы с коммуникацией типа точка-точка, сек | Время работы программы с коллективная коммуникация, сек |
|---|---|---|
| 2 | 49.2 | 49.1 |
| 4 | 24.65 | 24.64 |
| 8 | 12.86 | 12.87 |
| 16 | 13.36 | 13.06 |
| 24 | 19.29 | 18.85 |

Время работы последовательной программы составило 97.49 секунд.

### Ускорение

Ускорение высчитывается по формуле Sp=T1/Tp, где T1 – время работы последовательной программы. Tp – время работы параллельной программы на 2, 4, 16, 24 ядрах.



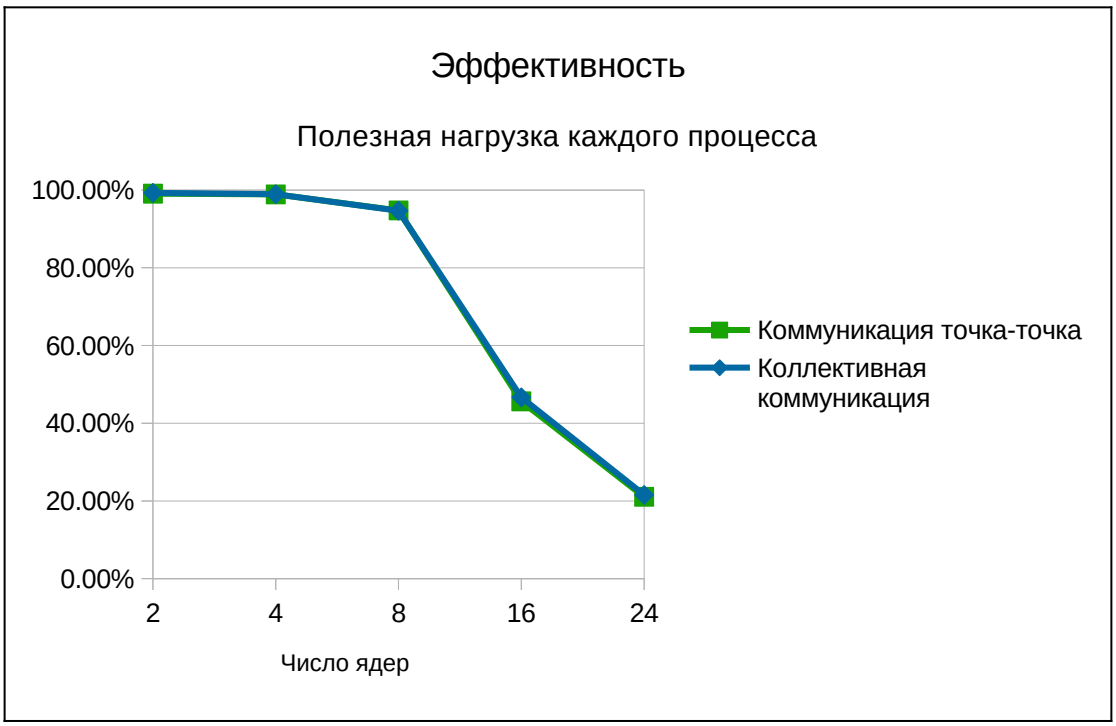

До 8 ядер ускорение интуитивно ожидаемое: если вычисление происходит на 2, 4 или 8 ядрах, то и времени будет тратиться в 2, 4, или 8 раз меньше соответственно. Однако при дальнейшем увеличении числа ядер, ускорение убывает. Это связано с тем, что полезная нагрузка (вычисление суммы) вытесняется нагрузкой бесполезной – коммуникацией процессов, а именно: **при увеличении числа процессов, увеличивается и количество передаваемых данных**.

Долю полезной нагрузки можно проследить на графике эффективности, рассмотренной в следующем параграфе.

## Эффективность

Эффективность Ep = Sp / p * 100%.

Эффективность отражает какой процент полезной нагрузки выполняет одно ядро. Из графика видно, что после 8 ядер эффективность стремительно падает. 20% на 24 ядрах означает, что каждое ядро лишь 20% времени работает над непосредственно вычислением, а остальные 80% занимается коммуникацией.




Из этих результатов можно сделать следующий вывод: бездумное увеличение числа ядер для выполнения программы не всегда будет ускорять программу, а напротив, может даже замедлить.

Также на обоих графиках нет значимой разницы между программы с коммуникацией типа точка-тока и с коллективной коммуникацией.

# ЗАКЛЮЧЕНИЕ

В результате лабораторной работы были написаны программы, использующие коммуникации типа точка-точка и коллективную коммуникацию. Также была выявлена закономерность — ускорение не обязательно растёт с увеличением числа ядер. Начиная с какого-то момента полезная нагрузка на ядро вытесняется коммуникацией между процессами. Поэтому нужно тестировать параллельные программы на разных конфигурациях — избыточный параллелизм может навредить производительности.

# Приложение 1. Код последовательной программы

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SEED 1000

long long calculate(const int* x, const int* y, int length) {
    long long s = 0;
    for (int i = 0; i < length; i++) {
        for (int j = 0; j < length; j++) {
            s += x[i] * y[j];
        }
    }
    return s;
}
// comment
int* create_vector(int length) {
    int* vector = (int*) malloc(sizeof(int) * length);
    if (!vector) return NULL;
    for (int i = 0; i < length; i++) { vector[i] = rand() %
10; }
    return vector;
}

void delete_vector(int** v) {
    if (*v) free(*v);
    *v = NULL;
}

int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);
    srand(SEED);
    if (argc != 2) {
        puts("Specify the vector length");
        puts("Usage:");
        printf("    %s [vector length]\n", argv[0]);
        MPI_Finalize();
        return 0;
    }

    const int length = atoi(argv[1]);

    int *x = create_vector(length);
    int *y = create_vector(length);
```

```
    double timer = MPI_Wtime();

    long long result = calculate(x, y, length);

    timer = MPI_Wtime() - timer;
    printf("%lld %.2lf\n", result, timer);

    delete_vector(&x); delete_vector(&y);

    MPI_Finalize();
    return 0;
}
```

# Приложение 2. Код программы с коммуникацией типа точка-точка

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define SEED 1000

#define LOG(format, ...) printf("%d: " format "\n", rank,
##__VA_ARGS__)
void LOG_VECTOR(int rank, const char* name, int* x, int
length);

const int TAG_PART_OF_FIRST = 12211;
const int TAG_FULL_SECOND = 12345;
const int TAG_RESULT = 99999;


long long calculate(const int* x, const int* y, int x_length,
int y_length) {
    long long s = 0;
    for (int i = 0; i < x_length; i++) {
        for (int j = 0; j < y_length; j++) {
            s += x[i] * y[j];
        }
    }
    return s;
}

int* create_vector(int legnth);
void delete_vector(int** v);

// calculate length of 1st vector that process with this rank
will handle
int part_length(int length, int rank, int size) {
    return (rank == size - 1) ? length / size + length % size
: length / size;
}

void process_rank_zero(int length, int x_part_length, int
size) {
    int rank = 0;
    double timer = MPI_Wtime();

    // init vectors
```

```
    int* x = create_vector(length);
    int* y = create_vector(length);

    // --------------- send vectors ---------------------//
    for (int i = 1; i < size; i++) {
        MPI_Send(y, length, MPI_INT, i, TAG_FULL_SECOND,
MPI_COMM_WORLD);
        int length_send = part_length(length, i, size);
        MPI_Send(x + i*x_part_length, length_send, MPI_INT,
i, TAG_PART_OF_FIRST, MPI_COMM_WORLD);
    }
    // -------------- calculate the part ----------------//
    long long sum = calculate(x, y, x_part_length, length);

    // ---------------- merge (recieve) results
--------------------//
    for (int i = 1; i < size; i++) {
        long long recieve;
        MPI_Recv(&recieve, 1, MPI_LONG_LONG, i, TAG_RESULT,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        sum += recieve;
    }


    timer = MPI_Wtime() - timer;
    printf("%lld %.2lf\n", sum, timer);

    delete_vector(&x);
    delete_vector(&y);
}


void process_rank_non_zero(int length, int x_part_length, int
rank, int size) {
    // --------------- recieve vectors -------------------//
        // ---------------- 2nd vector -------------------
        int* y = (int*) malloc(sizeof(int) * length);
        MPI_Recv(y, length, MPI_INT, 0, TAG_FULL_SECOND,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        // ---------------- 1st vector -----------------
        int* x = (int*) malloc(sizeof(int) * x_part_length);
        MPI_Recv(x, x_part_length, MPI_INT, 0,
TAG_PART_OF_FIRST, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        // ----------------- calculate
-------------------- //
        long long sum = calculate(x, y, x_part_length,
length);
```

```
        // --------------- merge to rank0
----------------- //
        MPI_Send(&sum, 1, MPI_LONG_LONG, 0, TAG_RESULT,
MPI_COMM_WORLD);
        free(x);
}


int main(int argc, char **argv) {
    srand(SEED);
    MPI_Init(&argc, &argv);

    if (argc != 2) {
        puts("Specify the vector length");
        puts("Usage:");
        printf("    %s [vector length]\n", argv[0]);
        MPI_Finalize();
        return 0;
    }

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    const int length = atoi(argv[1]);
    const int x_length = part_length(length, rank, size);

    if (rank == 0) {
        process_rank_zero(length, x_length, size);
    }
    else {  // can't enter this block if size == 1 becuse
(rank < size) => (rank == 0)
        process_rank_non_zero(length, x_length, rank, size);
    }

    MPI_Finalize();
    return 0;
}



// ============== Vector functions ===============

int* create_vector(int length) {
    int* vector = (int*)malloc(sizeof(int) * length);
    if (!vector) return NULL;
    for (int i = 0; i < length; i++) { vector[i] = rand() %
10; }
    return vector;
```

```
}

void delete_vector(int** v) {
    if (*v) free(*v);
    *v = NULL;
}
```

# Приложение 3. Код программы с коллективной коммуникацией

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define SEED 1000

#define LOG(format, ...) printf("%d: " format "\n", rank,
##__VA_ARGS__)
void LOG_VECTOR(int rank, const char* name, int* x, int
length);

const int TAG_PART_OF_FIRST = 12211;
const int TAG_FULL_SECOND = 12345;
const int TAG_RESULT = 99999;


long long calculate(const int* x, const int* y, int x_length,
int y_length) {
    long long s = 0;
    for (int i = 0; i < x_length; i++) {
        for (int j = 0; j < y_length; j++) {
            s += x[i] * y[j];
        }
    }
    return s;
}

// calculate length of 1st vector that process with this rank
will handle
int part_length(int length, int rank, int size) {
    return (rank == size - 1) ? length / size + length % size
: length / size;
}

int* create_vector(int legnth);
void delete_vector(int** v);
void fill_vector(int *vector, int length);



int main(int argc, char **argv) {
    srand(SEED);
    MPI_Init(&argc, &argv);
```

```
    if (argc != 2) {
        puts("Specify the vector length");
        puts("Usage:");
        printf("    %s [vector length]\n", argv[0]);
        MPI_Finalize();
        return 0;
    }

    int rank;
    int size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);


    const int length = atoi(argv[1]);

    double timer = MPI_Wtime();

    // init vectors
    int*global_x = NULL;
    int *global_y = (int*) malloc(sizeof(int) *length);
    if (rank == 0) {
        global_x = (int*) malloc(sizeof(int) * length);
        fill_vector(global_x, length);
        fill_vector(global_y, length);

        if (size == 1) {
            long long sum = 0;
            sum = calculate(global_x, global_y, length,
length);
            timer = MPI_Wtime() - timer;
            printf("%lld, %lf\n", sum, timer);
            delete_vector(&global_x);
delete_vector(&global_y);
            MPI_Finalize();
            return 0;
        }
    }



    // ------------------- size > 1 --------------------

    int* displs = (int*) malloc(sizeof(int) * size);
    int* sendcounts = (int*) malloc(sizeof(int) * size);

    displs[0] = 0;
    sendcounts[0] = length / size + length % size;
```

```
    for (int i = 1; i < size; i++) {
        displs[i] = displs[i - 1] + sendcounts[i - 1];
        sendcounts[i] = length / size;
    }

    int* local_x = (int*) malloc(sizeof(int) *
sendcounts[rank]);
    MPI_Bcast(global_y, length, MPI_INT, 0, MPI_COMM_WORLD);

    MPI_Scatterv(global_x, sendcounts, displs, MPI_INT,
        local_x, sendcounts[rank], MPI_INT, 0,
MPI_COMM_WORLD);

    long long local_sum = calculate(local_x, global_y,
sendcounts[rank], length);
    long long global_sum = 0;
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_LONG_LONG,
MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        timer = MPI_Wtime() - timer;
        printf("%lld %.2lf\n", global_sum, timer);
    }

    free(displs);
    free(sendcounts);
    delete_vector(&local_x);
    delete_vector(&global_y);
    delete_vector(&global_x);
    MPI_Finalize();
    return 0;
}



// ============== Vector functions ===============

void fill_vector(int *vector, int length) {
    for (int i = 0; i < length; i++) { vector[i] = rand() %
10; }
}

int* create_vector(int length) {
    int* vector = (int*)malloc(sizeof(int) * length);
    if (!vector) return NULL;
    for (int i = 0; i < length; i++) { vector[i] = rand() %
10; }
    return vector;
}
```

```
void delete_vector(int** v) {
    if (*v) free(*v);
    *v = NULL;
}

void LOG_VECTOR(int rank, const char* name, int* x, int
length) {
    printf("%d: %s [", rank, name);
        for (int i = 0; i < length; i++) {
            printf("%d ", x[i]);
        }
        printf("]\n");
}
```

# Приложение 4. Скрипт для тестирования

```
#!/bin/bash

# Check args count
if [ "$#" -ne 2 ]; then
    echo "Error: Need 2 arguments" >&2
    echo "    Usage: $0 [Vector length] [Num of tests]"
    exit 1
fi

# Check if args is a digit
if ! [[ "$1" =~ ^[0-9]+$ && "$1" =~ ^[0-9]+$  ]]; then
    echo "Error: Length of vector and num of tests should be
digits" >&2
    exit 1
fi

VECTOR_LENGTH=$1
NUM_TESTS=$2
LOG_FILE=log.txt

echo ":: Clear log.txt"
cat /dev/null > log.txt
echo "PROCS | RESULT | TIME (sec)" >> $LOG_FILE

run_tests() {
    echo ":: Run tests"
        for file in *.out; do
            echo ":: Testing $file"
            echo "-------- ${file%.out} -------" >>
$LOG_FILE
            if [ "$file" = "oneproc.out" ]; then
                echo ":: oneproc.out found -> Use only 1
proc"
                procs_list="1"
            else
                procs_list="2 4 8 16 26"
            fi

            for procs in $procs_list; do
                echo -n "$procs " >> $LOG_FILE
                #echo "mpirun -np $procs --bind-to core
$file $VECTOR_LENGTH >> $LOG_FILE"
                mpirun -np $procs ./$file $VECTOR_LENGTH >>
$LOG_FILE
```

```
            done
        done

        echo ":: Done"
}

for ((i=0; i<$NUM_TESTS; i++)); do
    echo ":: ========= Try $((i+1)) of $NUM_TESTS =========="
    run_tests
    echo ":: Move log.txt to log_$((i+1)).txt"
    mv log.txt log_$((i+1)).txt
done
```